

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 28

Инв. № подл.	Подп. и дата
Взам. инв. №	Инв. № дубл.
Подп. и дата	Подп. и дата

Выполнил: Друговская Е.М.
Группа: А-09-23
Вариант: 3

Проверил: Ключанский А.А.

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 3 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1530 байт, выработка stroba Y_1 длительностью 215 мс, выработка сигнала Y_2 со стробом 1899 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1 Общие сведения.....	4
2 Функциональное назначение.....	5
2.1 Классы решаемых задач.....	5
2.2 Назначение программы.....	5
2.3 Сведения о функциональных ограничениях на применение.....	5
3 Описание логической структуры.....	6
3.1 Алгоритм программы.....	6
3.2 Структура программы с описанием функций составных частей и связи между ними	18
3.3 Связи программы с другими программами.....	19
4 Входные данные.....	20
4.1 Клавиатура.....	20
4.2 Сигнал X2.....	20
5 Выходные данные.....	21
5.1 Характер и организация выходных данных.....	21
5.2 Формат, описание и способ кодирования выходных данных.....	21
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	23
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	24
ПРИЛОЖЕНИЕ А. Листинги программ.....	25

1 Общие сведения

Обозначение и наименование программы — «Программа управления объектом», вариант 3, группа А-09-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Программы разработаны в интегрированной среде Keil μ Vision V5 (пакет PK51 для микроконтроллеров семейства 8051): трансляция — ассемблером A51, компоновка — BL51, отладка — встроенным симулятором среды.

2 Функциональное назначение

2.1 Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2 Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — вывод символа по коду на индикатор с гашением через T_3 (прерывание таймера).

2.3 Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 1899 мкс на выводе P1.4;
- выходной сигнал Y_1 — строб низкого уровня длительностью 215 мс на выводе P1.3;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более $T_3 = 2,430$ с.

3 Описание логической структуры

3.1 Алгоритм программы

После сброса программа один раз проходит инициализацию (рисунок 1): указатель стека ставится на 07h, сигналом CS_EN = 1 включается дешифратор адреса, стробы Y1 и Y2 переводятся в пассивную единицу, все три столбца клавиатуры опускаются в ноль, индикатор гасится, буфер объявляется пустым (голова и хвост на 8000h, флаг пустоты взведён), оба таймера настраиваются в 16-разрядный режим и разрешаются прерывания. Основной цикл (рисунок 2) только следит за буфером: пока X1 не равно нулю и в буфере есть отсчёты, он берёт очередной отсчёт X2 по хвосту, считает $Y2 = (12 + X1 + X2) \bmod 256$ и выдаёт его со стробом; при X1 = 0 он один раз выдаёт нулевое Y2.

Всё, что приходит снаружи, обрабатывается в прерываниях. Нажатие клавиши даёт INT0: обработчик опрашивает матрицу 3×4 (рисунок 3) и по коду запоминает X1 и показывает цифру, запускает строб Y1 или очищает буфер (рисунок 4). Строб X2 от внешнего устройства приходит на INT1 по спаду; отсчёт к этому моменту ещё только записывается правым портом в ячейку обмена 9FFFh (ОЗУ фиксирует его по фронту строба), поэтому обработчик сначала ждёт, пока P3.3 вернётся в 1, затем читает ячейку левым портом и кладёт отсчёт в кольцо (рисунки 5 и 6). Длительности отсчитывают таймеры: Timer0 с тиком 5 мс заканчивает строб Y1 через 43 тика и гасит индикатор через 486 тиков (рисунок 10), Timer1 одной загрузкой отмеряет строб Y2 (рисунок 13).

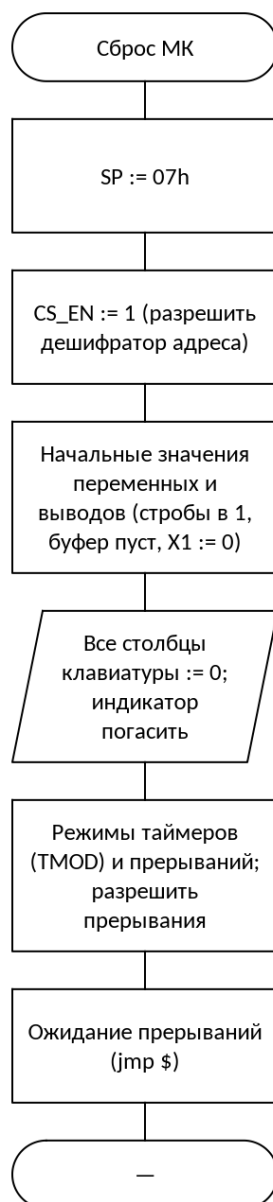


Рисунок 1 — Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; SP := 07h; CS_EN := 1 (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, X1 := 0); обмен с устройством: все столбцы клавиатуры := 0; индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp \$).

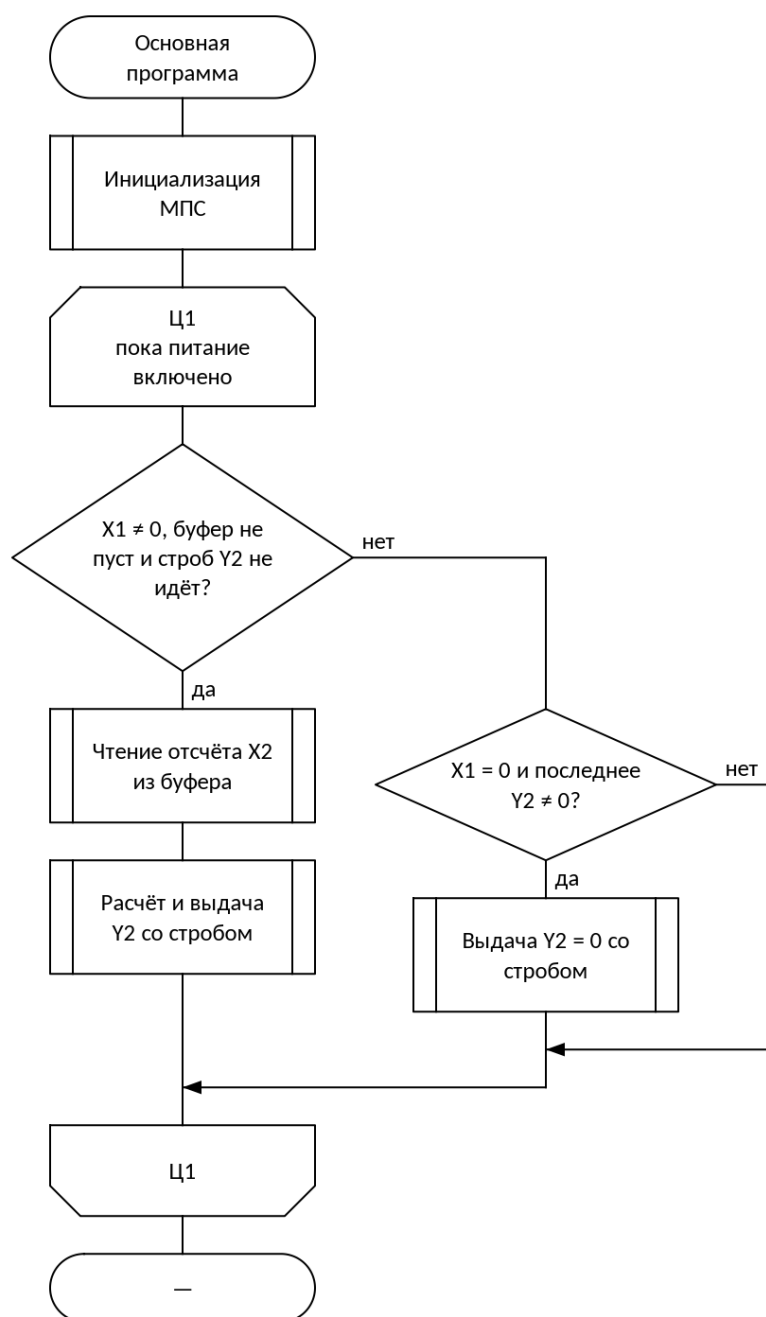


Рисунок 2 — Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие « $X1 \neq 0$, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие « $X1 = 0$ и последнее $Y2 \neq 0$ »: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

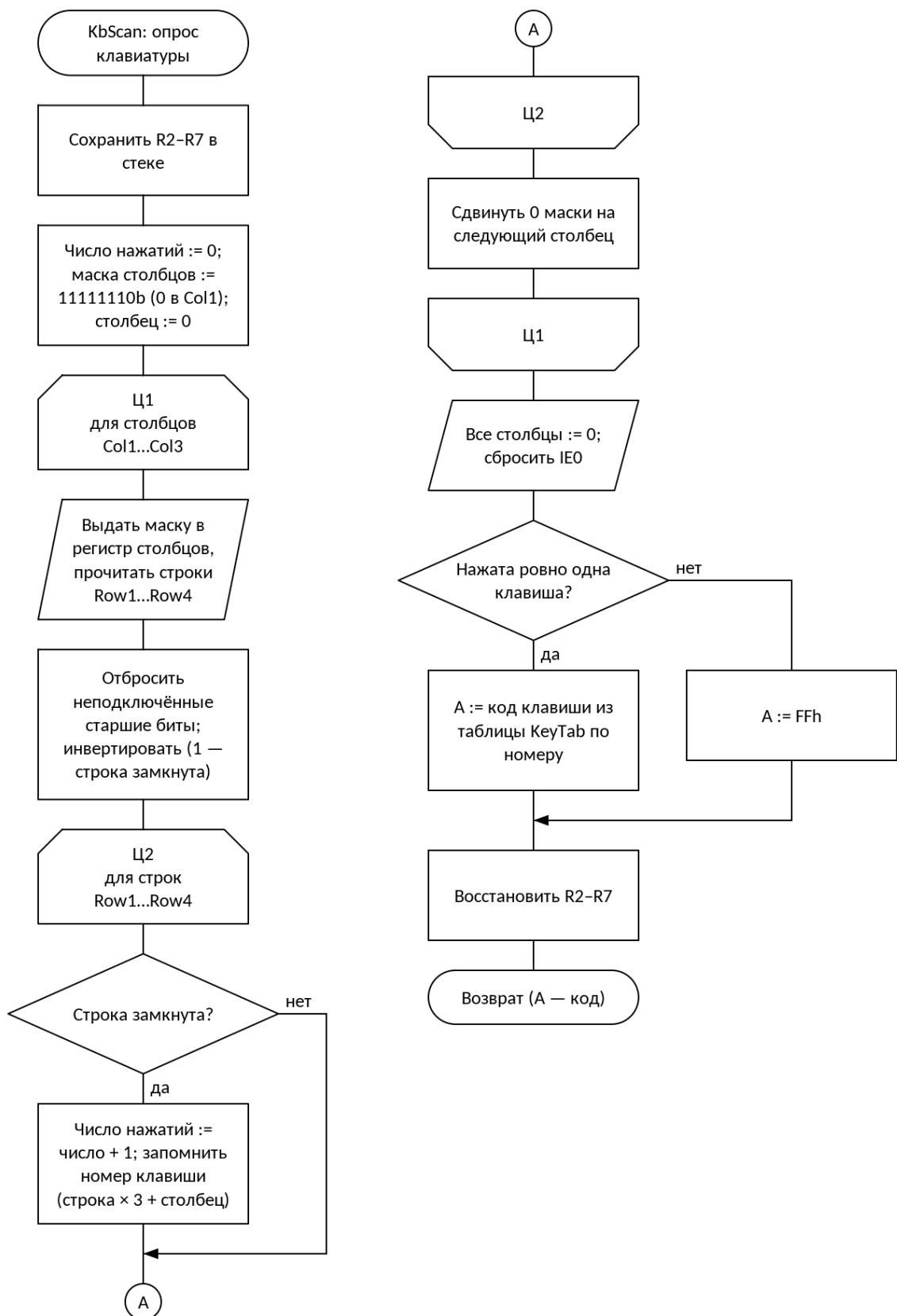


Рисунок 3 — Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Процедура KbScan опускает в ноль по очереди один из трёх столбцов и каждый раз читает строки. Старшие биты прочитанного байта не подключены, поэтому они отбрасываются, а оставшиеся четыре инвертируются, чтобы единица означала замкнутую строку. Каждую замкнутую строку процедура считает и запоминает номер клавиши как «строка × 3 + столбец». После перебора все столбцы снова опускаются, иначе следующее нажатие не даст прерывания,

и сбрасывается флаг IE0, который взвёлся, пока столбцы переключались. Если найдено ровно одно нажатие, номер переводится в код по таблице KeyTab, иначе возвращается FFh.

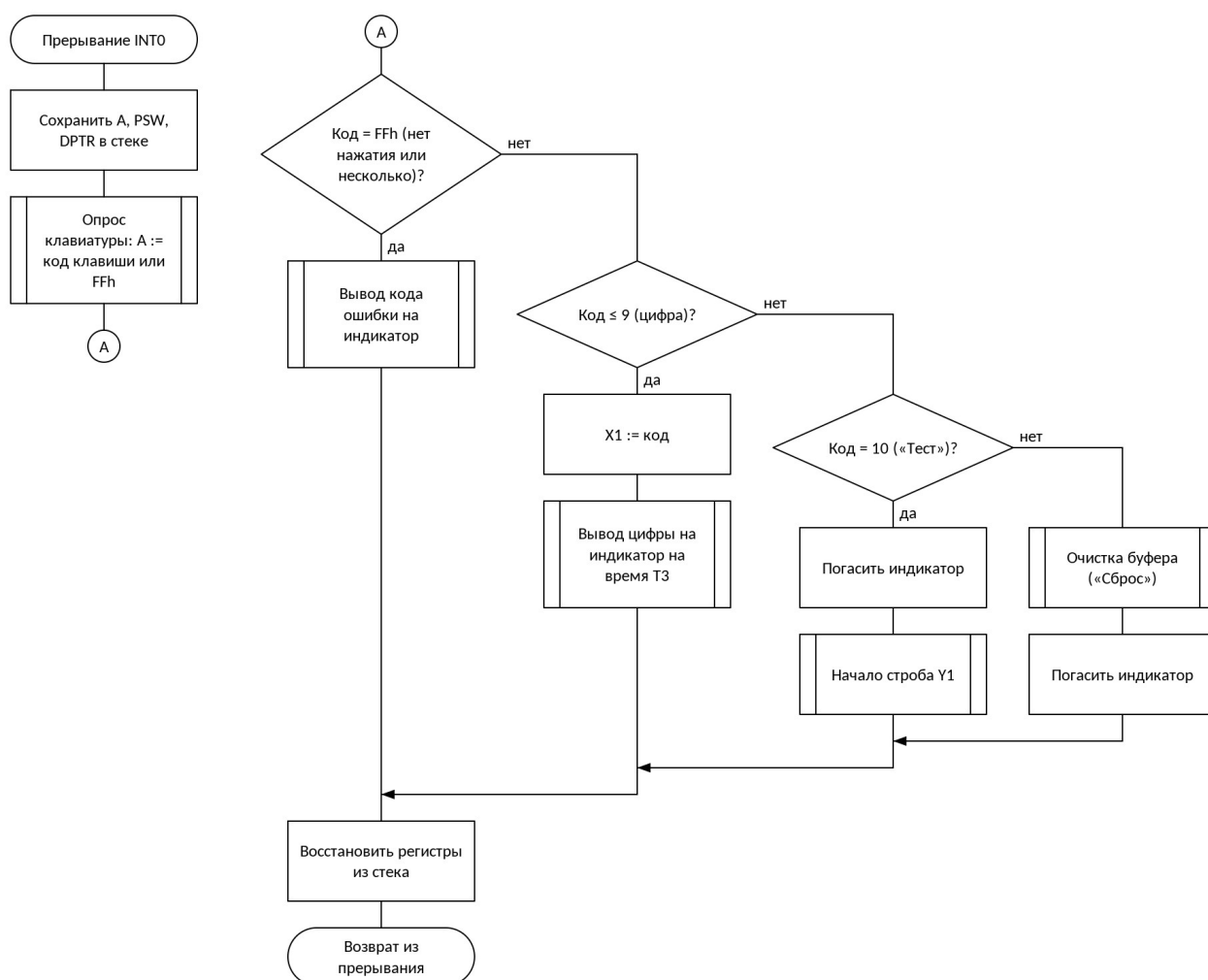


Рисунок 4 — Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.



Рисунок 5 — Схема алгоритма: Запись отсчёта в кольцевой буфер

Процедура BufWrite сначала проверяет, не заполнено ли кольцо: это случай, когда голова совпала с хвостом, а флаг пустоты сброшен. Тогда хвост сдвигается на одну ячейку вперёд, самый старый отсчёт пропадает и ставится флаг переполнения — то есть кольцо вмещает все 1530 отсчётов (политика V). Затем отсчёт записывается левым портом по адресу

головы, голова сдвигается, флаг пустоты сбрасывается. Переход на следующую ячейку вынесен в отдельную процедуру PtrStep: она увеличивает адрес и после 85F9h возвращает его на 8000h, сравнивая оба байта.

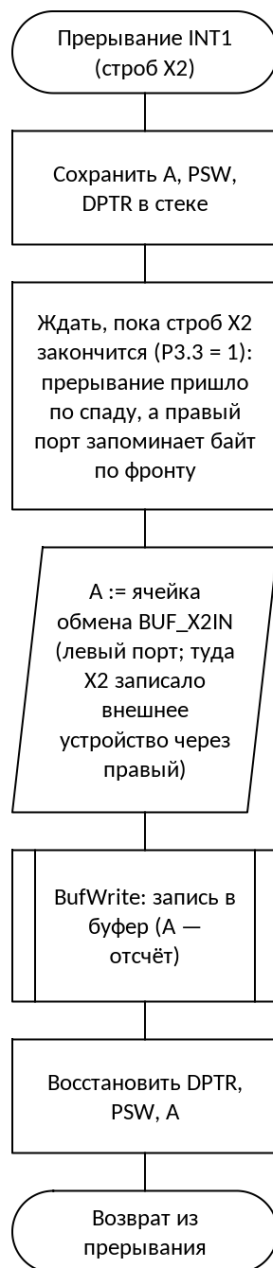


Рисунок 6 — Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Обработчик INT1 вызывается по спаду строба X2, но правый порт IDT7005 запоминает байт только по его фронту, поэтому первым делом обработчик крутится, пока на P3.3 снова не появится единица. После этого в ячейке обмена 9FFFh гарантированно лежит новый отсчёт, и обработчик читает его левым портом командой movx. Прочитанный байт передаётся процедуре записи в кольцо (рисунок 5), которая сама ведёт голову, хвост и флаги. Регистры A, PSW и DPTR сохраняются в стеке, потому что прерывание может прийти в любой точке основной программы.

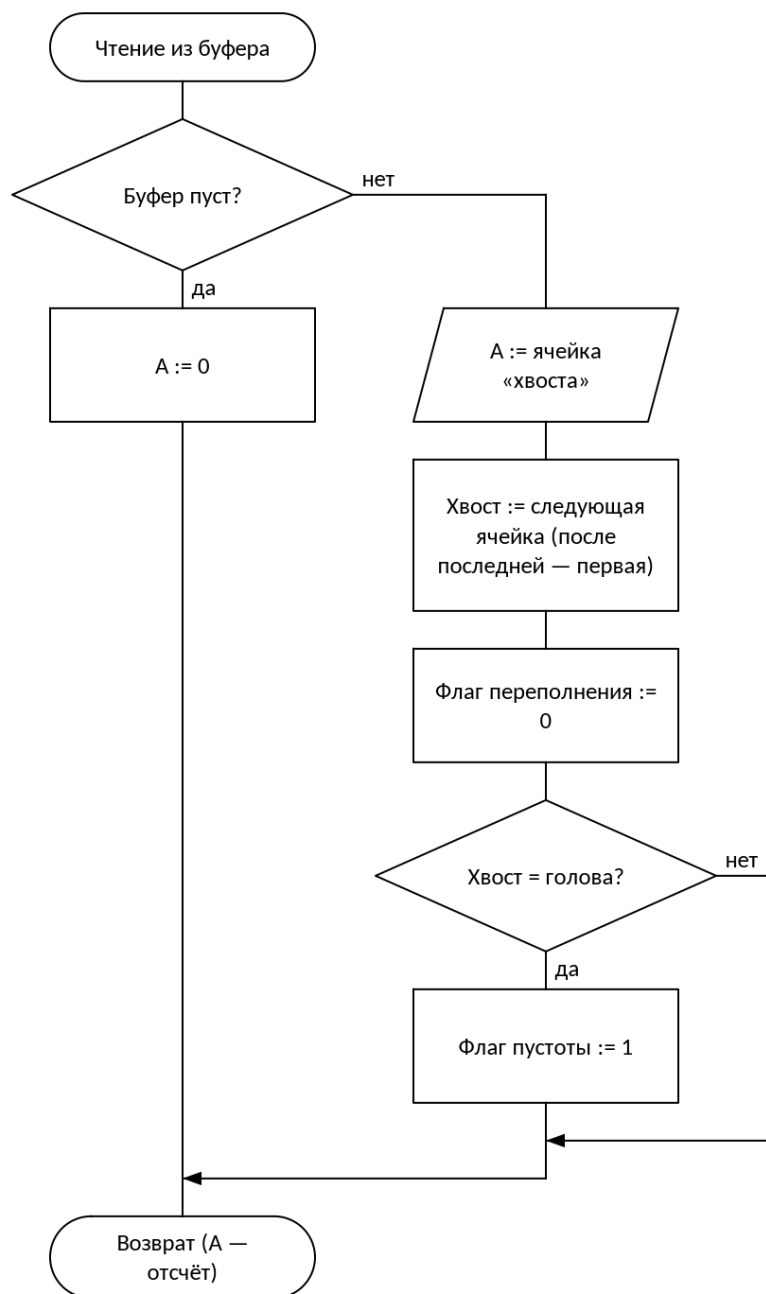


Рисунок 7 — Схема алгоритма: Чтение отсчёта из кольцевого буфера

Схема «Чтение отсчёта из кольцевого буфера» выполняется так: вход — «Чтение из буфера»; проверяется условие «Буфер пуст»: если да — $A := 0$, иначе — обмен с устройством: $A := \text{ячейка «хвоста»}$, $\text{хвост} := \text{следующая ячейка (после последней — первая)}$, $\text{флаг переполнения} := 0$, проверяется условие «Хвост = голова»: если да — $\text{флаг пустоты} := 1$; в конце — возврат (A — отсчёт).

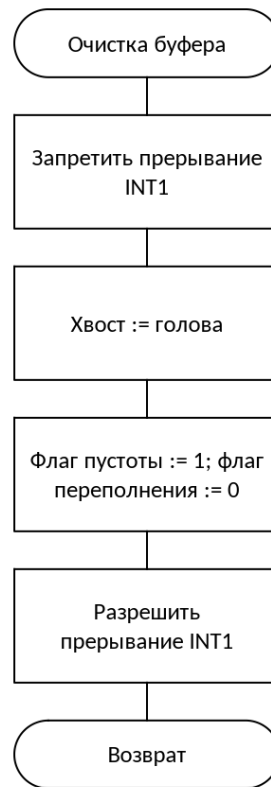


Рисунок 8 — Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

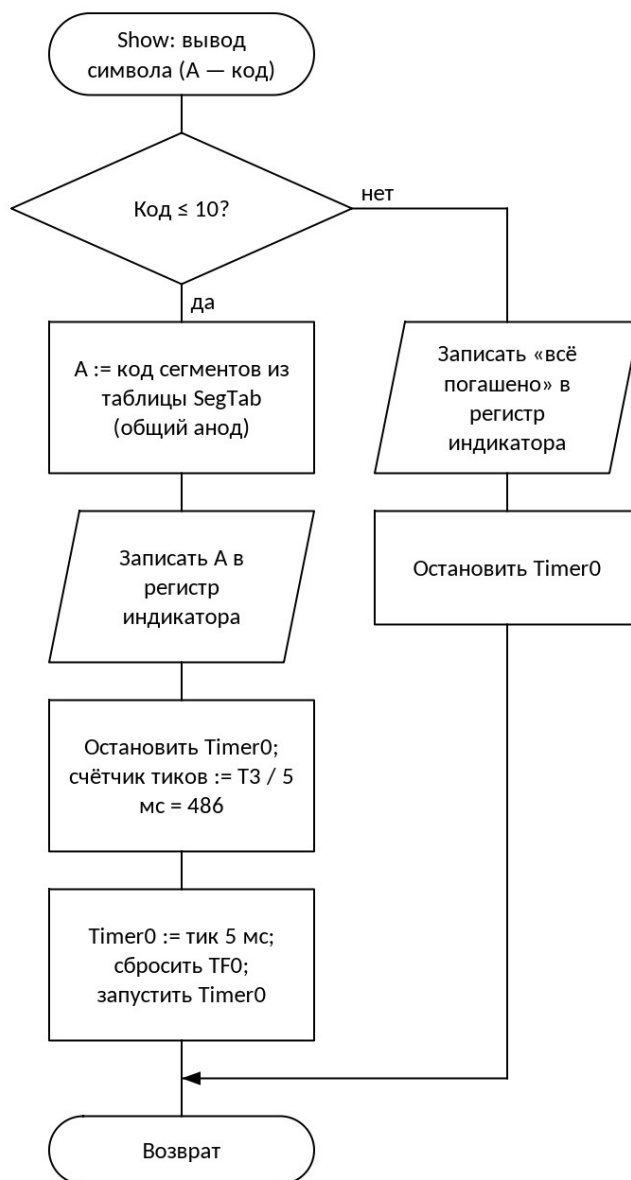


Рисунок 9 — Схема алгоритма: Вывод символа на индикатор

Процедура Show получает код символа в аккумуляторе. Коды 0–10 переводятся по таблице SegTab сразу в коды сегментов для индикатора с общим анодом (сегмент горит при нуле) и записываются в регистр индикатора. После этого отсчёт T3 начинается заново: Timer0 останавливается, счётчик тиков получает значение 486 (2430 мс по 5 мс), таймер загружается на первый тик и запускается — поэтому новый символ всегда горит полное T3. Для кодов 11 и больше индикатор сразу гасится, а таймер останавливается, так как ждать нечего.

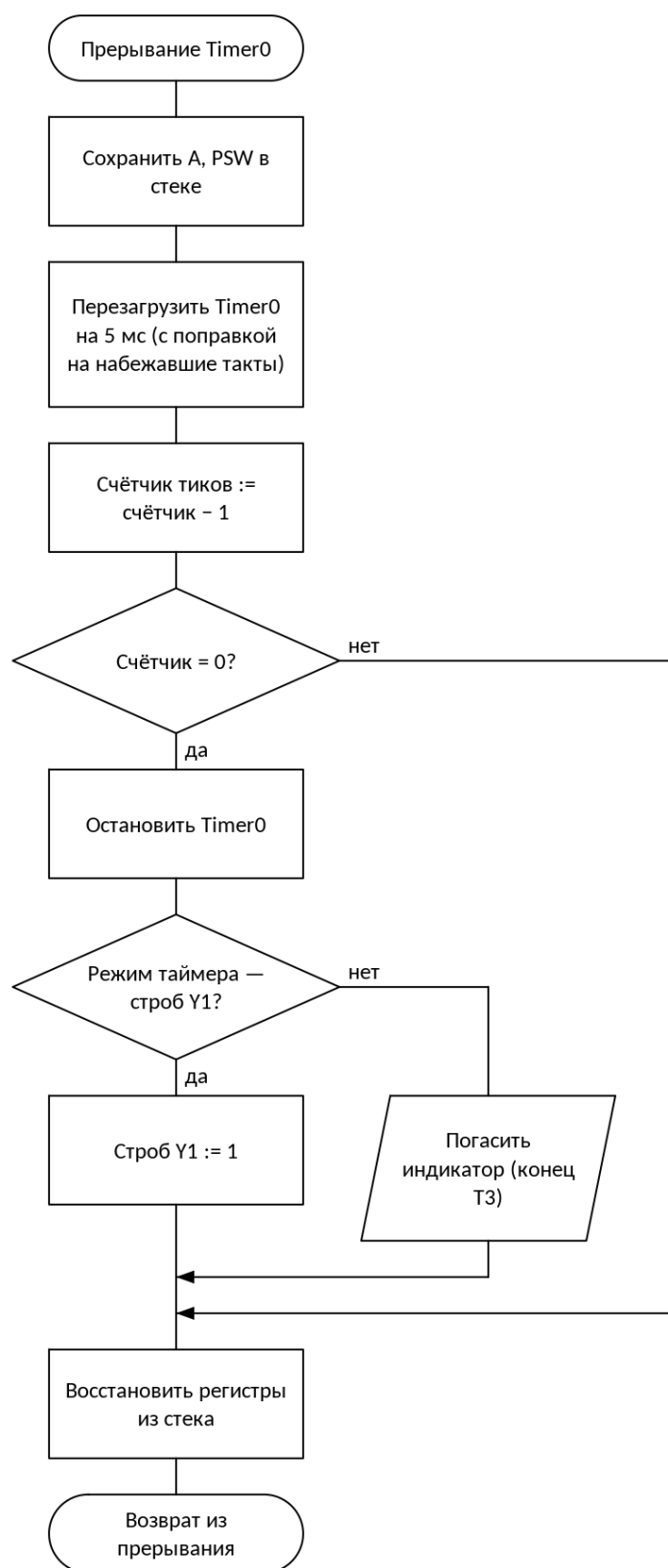


Рисунок 10 — Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Таймер 0 работает в 16-разрядном режиме и сам не перезагружается, поэтому обработчик на каждом тике заново загружает его на 5 мс. К загружаемому значению прибавляется то, что таймер успел насчитать после переполнения, плюс семь циклов, пока он стоит, — так задержка входа в прерывание не накапливается за 486 тиков. Затем уменьшается 16-разрядный счётчик тиков, и когда он доходит до нуля, таймер останавливается и индикатор гаснет (в полной системе тот же таймер заканчивает и строб Y1).



Рисунок 11 — Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; счётчик тиков := $T1 / 5$ мс = 43; Timer0 := тик 5 мс; строб Y1 := 0; запустить Timer0; в конце — возврат.

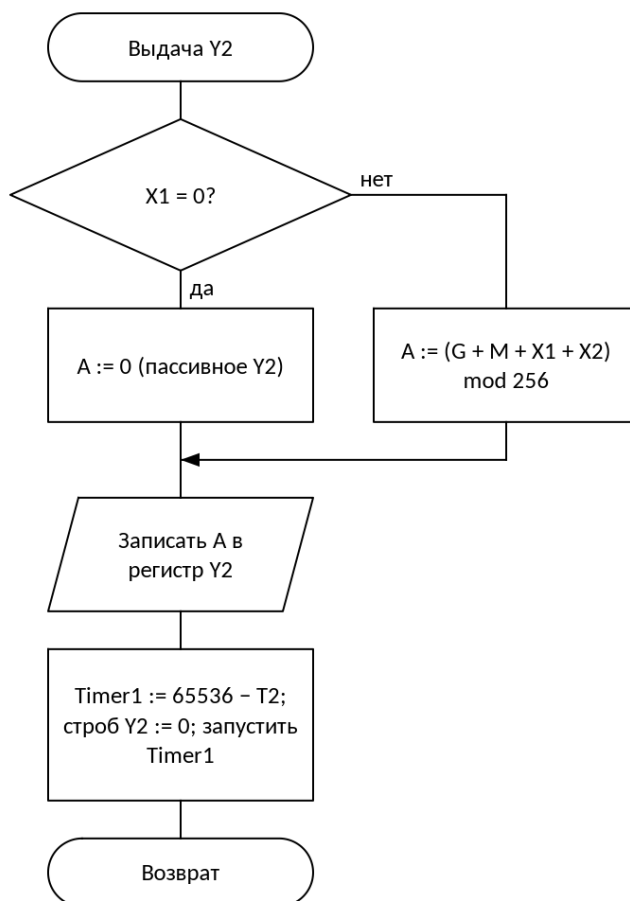


Рисунок 12 — Схема алгоритма: Расчёт и выдача сигнала Y2

Схема «Расчёт и выдача сигнала Y2» выполняется так: вход — «Выдача Y2»; проверяется условие « $X1 = 0$ »: если да — $A := 0$ (пассивное Y2), иначе — $A := (G + M + X1 + X2) \bmod 256$; обмен с устройством: записать A в регистр Y2; Timer1 := 65536 - T2; строб Y2 := 0; запустить Timer1; в конце — возврат.



Рисунок 13 — Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Схема «Обработчик прерывания Timer1: конец строба Y2» выполняется так: вход — «Прерывание Timer1»; остановить Timer1; строб Y2 := 1; в конце — возврат из прерывания.

3.2 Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы клавиатуры 3x4 по столбцам (вызывается из обработчика INT0).

Входные данные: нет.

Выходные данные: А — код нажатой клавиши (0–9, «Т» = 10, «С» = 11) или 0FFh, если нажато не ровно одна клавиша. Столбцы после опроса снова все в 0, IE0 сброшен.

Используемые регистры: А, В, DPTR, PSW (R2–R7 сохраняются в стеке).

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись отсчёта в кольцевой буфер в ячейку «головы». Если буфер уже полон (голова догнала хвост, буфер не пуст), самый старый отсчёт теряется: хвост уходит на следующую ячейку, F_OVF = 1.

Входные данные: А — отсчёт.

Выходные данные: отсчёт в буфере, голова сдвинута, F_EMPTY = 0; F_OVF = 1 при затирании.

Используемые регистры: DPTR, PSW (А сохраняется).

3.2.3 Подпрограмма PtrStep (программа 2)

Назначение: переход указателя буфера на следующую ячейку кольца.

Входные данные: DPTR — адрес ячейки буфера.

Выходные данные: DPTR — следующая ячейка; после последней (BUF_END - 1) — BUF_START.

Используемые регистры: PSW (А сохраняется).

3.2.4 Подпрограмма Show (программа 3)

Назначение: вывод символа на индикатор (общий анод) на время T3; гасит его обработчик Timer0.

Входные данные: А — код по табл. 2 T3: 0–9 — цифра, 10 — «Е», больше — погасить.

Выходные данные: символ в регистре индикатора; для 0–10 заново запущен отсчёт ТЗ.

Используемые регистры: А, DPTR, PSW.

3.2.5 Обработчик прерывания Timer0 — T0Isr (программа 3)

Назначение: тик Timer0 (5 мс): перезагрузка таймера с поправкой, счёт тиков ТЗ, по окончании ТЗ — индикатор гаснет и таймер останавливается.

Входные данные: ТЗCnt.

Выходные данные: ТЗCnt уменьшен.

Используемые регистры: ничего (А, PSW, DPTR сохраняются).

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (215 мс) и T_3 (2430 мс), тик 5 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (1899 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y5	0A000h–0BFFFh
Клавиатура	Y7	0E000h–0FFFFh
Буфер (IDT7005)	Y4	8000h–9FFFh
Регистр Y2	Y6	0C000h–0DFFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	44h, 45h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	46h, 47h	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	20h.3 (бит 03h)	флаг пустоты буфера
F_OVF	21h.4 (бит 0Ch)	флаг переполнения буфера
ТЗCnt	30h	переменная программы 3

3.3 Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4 Входные данные

4.1 Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 0E000h, разряды D0...D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 T3) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2 Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1530 байт (адреса 8000h–85F9h).

5 Выходные данные

5.1 Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше $T_3 = 2,430$ с; при ошибке — код ошибки с горящей точкой dp. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 8000h–85F9h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 215 мс по клавише «Тест». $Y_2 = (9 + 3 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 1899 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{\text{мц}} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:ТЛ1 = 65536 – $T_2 / T_{\text{мц}} = 65536 - 1899 = 63637 = 0F895h$ (строб Y_2 , таймер 1)

ТН0:ТЛ0 = 65536 – 5 мс / $T_{\text{мц}} = 65536 - 5000 = 60536 = 0EC78h$ (тик 5 мс, таймер 0)

$N_{T1} = T_1 / 5 \text{ мс} = 43$; $N_{T3} = T_3 / 5 \text{ мс} = 486$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

$t_{\text{min}} = V / f_{X2} = 1530 / 4 = 382,5$ с (6,4 мин).

Максимальная глубина стека для программы 1 — 10 байт (SP от 07h до 11h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2 Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dp; индикатор с общим анодом — сегмент горит при 0.

Таблица 5 — Коды сегментов индикатора (dp g f e d c b a)

Код	Символ	Двоичный	Шестн.
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«Е»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 6 — Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше ТЗ)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Таблица описывает реакцию полной системы. В написанных программах источники ошибок такие: процедура опроса клавиатуры возвращает FFh и при нескольких нажатых клавишах, и при отсутствии нажатия (в полной системе обработчик INT0 в ответ на FFh выводит код ошибки на индикатор, рисунок 4), а процедура записи в буфер при затирании непрочитанного отсчёта ставит флаг переполнения 21h.4 (код «3.»). Код «4.» относится к стробу Y_1 , который в трёх программах варианта не реализуется.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0C000h), строб — на выводе P1.4; сигнал Y_1 — на выводе P1.3.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Друговская ЕМ-код-1»

```
; Друговская Е.М., А-09-23, 3, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
DSEG AT 30h
; переменные не нужны: всё держится в регистрах на время опроса
CSEG AT 0h
    sjmp START
org 03h ; INT0 — нажатие клавиши: процедура вызывается отсюда
    call KbScan ; %proc%
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN ; включаем дешифратор, иначе ни одно устройство не выбирается
    mov DPTR, #ADR_KB
    clr A
    movx @DPTR, A ; все столбцы опущены: любое нажатие сразу опустит строку и даст INT0
    setb IT0 ; INT0 срабатывает по спаду
    setb EX0 ; прерывание от клавиатуры разрешено
    setb EA ; общее разрешение прерываний
    jmp $ ; %stop%

; KbScan — опрос матрицы клавиатуры 3x4 по столбцам (вызывается из обработчика INT0).
; Вход: нет.
; Выход: A — код нажатой клавиши (0-9, «Т» = 10, «С» = 11) или 0FFh, если нажато
; ; не ровно одна клавиша. Столбцы после опроса снова все в 0, IE0 сброшен.
; Портит: A, B, DPTR, PSW (R2-R7 сохраняются в стеке).
KbScan:
    push AR2
    push AR3
    push AR4
    push AR5
    push AR6
    push AR7
    mov R2, #0 ; сколько замкнутых контактов нашли
    mov R3, #0FFh ; номер найденной клавиши в таблице KeyTab
    mov R4, #0FEh ; маска столбцов: ноль стоит в Col1
    mov R5, #0 ; номер текущего столбца
    mov DPTR, #ADR_KB
KsCol:
    mov A, R4
    movx @DPTR, A ; опускаем только текущий столбец
    movx A, @DPTR ; читаем строки
    anl A, #(1 SHL KB_ROWS) - 1 ; старшие линии шины не подключены — отбрасываем
    xrl A, #(1 SHL KB_ROWS) - 1 ; теперь 1 означает «строка замкнута»
    mov R6, A ; замкнутые строки этого столбца
    mov A, R5
    mov R7, A ; номер клавиши в строке Row1 этого столбца
    mov B, #KB_ROWS ; проверим каждую строку
KsRow:
    mov A, R6
    rrc A ; очередная строка уходит в C
    mov R6, A
    jnc KsSkip ; эта строка не замкнута
    inc R2 ; ещё одно нажатие
    mov A, R7
    mov R3, A ; запоминаем, какая клавиша
KsSkip:
    mov A, R7
    add A, #KB_COLS ; в следующей строке номер больше на число столбцов
    mov R7, A
    djnz B, KsRow
    mov A, R4
    rl A ; ноль маски переезжает в следующий столбец
    mov R4, A
```

```

inc R5
cjne R5, #KB_COLS, KsCol
clr A
movx @DPTR, A ; все столбцы снова опущены — ждём следующего нажатия
clr IE0 ; при переборе столбцов линия INT0 дёргалась — лишний запрос гасим
mov A, #0FFh ; по умолчанию — ошибка
cjne R2, #1, KsDone ; ноль или несколько нажатий — оставляем 0FFh
mov A, R3
mov DPTR, #KeyTab
movc A, @A+DPTR ; номер клавиши -> её код по табл. 3 ТЗ
KsDone:
pop AR7
pop AR6
pop AR5
pop AR4
pop AR3
pop AR2
ret

; Коды клавиш по номеру (строка * 3 + столбец), раскладка 3x4 рис. 4 ТЗ.
KeyTab: DB 1, 2, 3
        DB 4, 5, 6
        DB 7, 8, 9
        DB 0, 10, 11
END

```

Программа 2 — файл «Друговская ЕМ-код-2»

```

; Друговская Е.М., А-09-23, 3, v1
$INCLUDE (vars.inc) ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; переменные не нужны: указатели и флаги буфера заданы в vars.inc
CSEG AT 0h
sjmp START
org 03h ; "заглушка" для INT0
nop
reti
org 0bh ; "заглушка" для Tmr0_ovf
nop
reti
org 13h ; "заглушка" для INT1
nop
reti
org 1bh ; "заглушка" для Tmr1_ovf
nop
reti
org 23h ; "заглушка" для UART
nop
reti
org 2bh
START:
; Инициализация МК *****
mov SP, #07h
setb PIN_CSEN ; дешифратор включён — буфер доступен по movx
mov HEAD_L, #LOW(BUF_START) ; писать будем с первой ячейки кольца
mov HEAD_H, #HIGH(BUF_START)
mov TAIL_L, #LOW(BUF_START) ; читать — тоже с первой: голова = хвост
mov TAIL_H, #HIGH(BUF_START)
setb F_EMPTY ; отсчётов в буфере нет
clr F_OVF ; ничего не потеряно
mov A, #55h ; пример отсчёта
call BufWrite ; %proc%
jmp $ ; %stop%

; BufWrite — запись отсчёта в кольцевой буфер в ячейку «головы».
; Если буфер уже полон (голова догнала хвост, буфер не пуст), самый старый
; отсчёт теряется: хвост уходит на следующую ячейку, F_OVF = 1.
; Вход: A — отсчёт.
; Выход: отсчёт в буфере, голова сдвинута, F_EMPTY = 0; F_OVF = 1 при затирании.
; Портит: DPTR, PSW (A сохраняется).
BufWrite:
push ACC ; отсчёт понадобится после проверок
jb F_EMPTY, BwPut ; пустой буфер полным быть не может
mov A, HEAD_L
cjne A, TAIL_L, BwPut ; голова не на хвосте — место есть
mov A, HEAD_H
cjne A, TAIL_H, BwPut
mov DPL, TAIL_L ; буфер полон: выбрасываем самый старый отсчёт
mov DPH, TAIL_H
call PtrStep
mov TAIL_L, DPL
mov TAIL_H, DPH
setb F_OVF ; отмечаем, что отсчёт потерян
BwPut:
mov DPL, HEAD_L ; адрес ячейки «головы»

```

```

mov DPH, HEAD_H
pop ACC
movx @DPTR, A          ; кладем отсчет левым портом IDT7005
call PtrStep
mov HEAD_L, DPL        ; голова — на следующую свободную ячейку
mov HEAD_H, DPH
clr F_EMPTY            ; теперь в буфере точно что-то есть
ret

; PtrStep — переход указателя буфера на следующую ячейку кольца.
; Вход: DPTR — адрес ячейки буфера.
; Выход: DPTR — следующая ячейка; после последней (BUF_END - 1) — BUF_START.
; Портит: PSW (A сохраняется).
PtrStep:
    push ACC
    inc DPTR
    mov A, DPL
    cjne A, #LOW(BUF_END), PsDone    ; не дошли до конца кольца
    mov A, DPH
    cjne A, #HIGH(BUF_END), PsDone
    mov DPTR, #BUF_START            ; вышли за последнюю ячейку — заворачиваем на начало
PsDone:
    pop ACC
    ret
END

```

Программа 3 — файл «Друговская ЕМ-код-3»

```

; Друговская Е.М., А-09-23, 3, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
T0_LOAD EQU (TICK_H SHL 8) + TICK_L + 7 ; загрузка тика 5 мс + 7 циклов, пока таймер стоит в обработке
DSEG AT 30h
T3Cnt: DS 2                  ; сколько тиков по 5 мс осталось до гашения (младший, старший байт)
CSEG AT 0h
    sjmp START
org 03h ; "заглушка" для INT0
    nop
    reti
org 0bh ; Timer0 — тик для отсчёта T3
    ljmp T0Isr
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN          ; дешифратор включён — регистр индикатора доступен
    mov DPTR, #ADR_IND
    mov A, #IND_OFF
    movx @DPTR, A          ; после сброса индикатор тёмный
    mov TMOD, #11h         ; оба таймера — 16-разрядные (режим 1)
    setb ET0               ; прерывание Timer0 — отсчёт T3
    setb EA
    mov A, #5              ; пример: показать «5»
    call Show ; %proc%
    jmp $ ; %stop%

; Show — вывод символа на индикатор (общий анод) на время T3; гасит его обработчик Timer0.
; Вход: A — код по табл. 2 T3: 0-9 — цифра, 10 — «Е», больше — погасить.
; Выход: символ в регистре индикатора; для 0-10 заново запущен отсчёт T3.
; Портит: A, DPTR, PSW.
Show:
    cjne A, #11, $+3        ; C = 1, если код меньше 11
    jnc ShBlank            ; 11 и больше — показывать нечего
    mov DPTR, #SegTab
    movc A, @A+DPTR        ; код символа -> сегменты
    mov DPTR, #ADR_IND
    movx @DPTR, A          ; символ загорается
    clr TR0                ; старый отсчёт (если шёл) отменяем
    clr ET0                ; счётчик меняем без помех от прерывания
    mov T3Cnt, #LOW(T3_TICKS)
    mov T3Cnt+1, #HIGH(T3_TICKS)
    mov TH0, #TICK_H        ; первый тик 5 мс
    mov TL0, #TICK_L
    clr TF0
    setb ET0
    setb TR0                ; пошёл отсчёт T3
    ret

```

```

ShBlank:
    mov A, #IND_OFF
    mov DPTR, #ADR_IND
    movx @DPTR, A          ; гасим индикатор сразу
    clr TR0                ; отсчёт ТЗ больше не нужен
    ret

; T0Isr — тик Timer0 (5 мс): перезагрузка таймера с поправкой, счёт тиков ТЗ,
; по окончании ТЗ — индикатор гаснет и таймер останавливается.
; Вход: T3Cnt. Выход: T3Cnt уменьшен. Портит: ничего (A, PSW, DPTR сохраняются).
T0Isr:
    push ACC
    push PSW
    clr TR0                ; на время перезагрузки таймер стоит
    mov A, TL0             ; TL0/TH0 уже набегали после переполнения — прибавляем к ним
    add A, #LOW(T0_LOAD)
    mov TL0, A
    mov A, TH0
    addc A, #HIGH(T0_LOAD)
    mov TH0, A
    setb TR0
    mov A, T3Cnt           ; T3Cnt := T3Cnt - 1 (16 бит)
    jnz T0Low
    dec T3Cnt+1

T0Low:
    dec T3Cnt
    mov A, T3Cnt
    orl A, T3Cnt+1
    jnz T0Exit            ; ТЗ ещё не кончилось
    clr TR0               ; ТЗ вышло — таймер больше не нужен
    push DPL
    push DPH
    mov DPTR, #ADR_IND
    mov A, #IND_OFF
    movx @DPTR, A         ; гасим индикатор
    pop DPH
    pop DPL

T0Exit:
    pop PSW
    pop ACC
    reti

; Сегменты для общего анода (0 — сегмент горит; D0..D7 = a..g, dp): цифры 0-9 и «Е».
SegTab: DB 0C0h, 0F9h, 0A4h, 0B0h, 099h
        DB 092h, 082h, 0F8h, 080h, 090h
        DB 086h

END

```